United States Patent	12400136
Kind Code	B2
Date of Patent	August 26, 2025
Inventor(s)	Yang; Wenzhuo et al.

Systems and methods for counterfactual explanation in machine learning models

Abstract

Embodiments described herein provide a two-stage model-agnostic approach for generating counterfactual explanation via counterfactual feature selection and counterfactual feature optimization. Given a query instance, counterfactual feature selection picks a subset of feature columns and values that can potentially change the prediction and then counterfactual feature optimization determines the best feature value for the selected feature as a counterfactual example.

Inventors: **Yang; Wenzhuo (Singapore, SG), Li; Jia (Mountain View, CA), Hoi; Chu Hong (Singapore, SG), Xiong; Caiming (Menlo Park, CA)**

Applicant: **Salesforce, Inc.** (San Francisco, CA)

Family ID: **1000008782570**

Assignee: **Salesforce, Inc. (San Francisco, CA)**

Appl. No.: **17/162967**

Filed: **January 29, 2021**

Prior Publication Data

Document Identifier	Publication Date
US 20220114464 A1	Apr. 14, 2022

Related U.S. Application Data

us-provisional-application US 63089473 20201008

Publication Classification

Int. Cl.: **G06N5/045** (20230101); **G06F18/21** (20230101); **G06F18/2113** (20230101); **G06F18/2413** (20230101); **G06F18/243** (20230101); **G06N5/01** (20230101); **G06N7/01** (20230101); **G06N20/00** (20190101)

U.S. Cl.:

CPC **G06N5/045** (20130101); **G06F18/2113** (20230101); **G06F18/217** (20230101); **G06F18/24147** (20230101); **G06F18/24323** (20230101); **G06N5/01** (20230101); **G06N7/01** (20230101); **G06N20/00** (20190101);

Field of Classification Search

CPC: G06N (5/045); G06N (5/01); G06N (7/01); G06N (20/00); G06N (3/045); G06N (3/047); G06N (3/088); G06N (20/20); G06F (18/2113); G06F (18/217); G06F (18/24147); G06F (18/24323); G06F (18/211)

References Cited

U.S. PATENT DOCUMENTS

Patent No.	Issued Date	Patentee Name	U.S. Cl.	CPC
2009/0254971	12/2008	Herz	726/1	G06Q 10/10
2010/0082513	12/2009	Liu	706/46	H04L 63/1458
2014/0025509	12/2013	Reisz	705/14.71	G06Q 30/0244
2019/0220772	12/2018	Schmidt	N/A	G06F 16/2453
2020/0090075	12/2019	Achin	N/A	G06Q 30/0201
2020/0097997	12/2019	Li	N/A	G06N 20/00
2020/0126126	12/2019	Briancon	N/A	G06N 20/20
2020/0401891	12/2019	Xu	N/A	G06N 3/08
2021/0064517	12/2020	Wong	N/A	G06F 11/3698
2021/0117508	12/2020	Chang	N/A	G06F 17/18
2021/0117784	12/2020	Flinn	N/A	G06F 18/2148

OTHER PUBLICATIONS

International Search Report and Written Opinion for PCT/US2021/053995, dated Feb. 2, 2022, 28 pages. cited by applicant

Mothilal et al., "Explaining Machine Learning Classifiers Through Diverse Counterfactual Explanations", Arxiv.Org, Cornell University Library, 201 Olin Library Cornell University Ithaca, NY 14853, May 19, 2019, 13 Pages, KP081546163. cited by applicant

Guidotti, et al., "Factual and Counterfactual Explanations for Black-Box Decision Making,", p. 2, Section II, under "Local Rule-Based Explanations of Black Box Decision System,", IEEE Intelligent Systems Journal, 2019, pp. 1-7. cited by applicant

---

Primary Examiner: Sheffield; Hope C

Attorney, Agent or Firm: Haynes and Boone, LLP

Background/Summary

(1) The present application is a nonprovisional of and claims priority under 35 U.S.C. 119 to U.S. provisional application No. 63/089,473, filed on Oct. 8, 2020. (2) The present application is related to U.S. nonprovisional application Ser. No. 17/162,931, filed on Jan. 29, 2021. (3) All of the aforementioned applications are hereby expressly incorporated by reference herein in their entirety.

TECHNICAL FIELD

(1) The present disclosure relates generally to machine learning models and neural networks, and more specifically, to a counterfactual explanation system via candidate counterfactual feature selection and optimization for machine learning models.

BACKGROUND

(2) Machine learning models are widely-applied in various applications, leading to promising results especially in computer vision, natural language processing, and recommendation systems. As the adoption of these models grows rapidly, their algorithmic decisions in healthcare, education and finance can potentially have a huge social impact on the public. However, many machine learning models, and in particular deep learning models, generally work as black-box models, providing little visibility and knowledge on why certain decisions are made. The lack of explainability remains a key concern in many applications and thus hampers further implementation of machine learning/artificial intelligence (AI) systems with full trust and confidence.

Description

BRIEF DESCRIPTION OF THE DRAWINGS

(1) FIG. **1** shows a simplified diagram illustrating an overview of generating counterfactual explanations for a machine learning model, according to the embodiments described herein.
(2) FIG. **2** provides an overview of the counterfactual example generation module shown in FIG. **1**, according to embodiments described herein.
(3) FIG. **3** is a simplified diagram of a computing device for generating counterfactual explanations, according to some embodiments.
(4) FIG. **4** provides an example logic flow diagram illustrating a method of counterfactual example generation, according to embodiments described herein.
(5) FIG. **5** provides an example pseudo-code segment illustrating an algorithm of the nearest neighbor search approach for generating counterfactual features, according to embodiments described herein.
(6) FIG. **6** provides an example pseudo-code segment illustrating an algorithm of building a recommendation model for generating counterfactual features, according to embodiments described herein.
(7) FIG. **7** provides an example pseudo-code segment illustrating an algorithm of generating counterfactual feature selections based on a recommendation model, according to embodiments described herein.
(8) FIG. **8** shows a simplified diagram illustrating the neural network model structure of a recommendation model for feature selection, according to embodiments described herein.
(9) FIG. **9** provides an example pseudo-code segment illustrating an algorithm of reinforcement learning based counterfactual feature optimization for generating a counterfactual example given selected counterfactual features, according to embodiments described herein.
(10) FIGS. **10**A-**10**B provide alternative example pseudo-code segments illustrating an algorithm of gradientless descent based counterfactual feature optimization for generating a counterfactual example given selected counterfactual features, according to embodiments described herein.
(11) FIG. **11** provides an example pseudo-code segment illustrating an algorithm of generating

diverse counterfactual examples, according to embodiments described herein.
(12) FIGS. **12**-**27** provide example data tables illustrating data experiment results of the counterfactual example generation, according to one embodiment.
(13) In the figures, elements having the same designations have the same or similar functions.
DETAILED DESCRIPTION
(14) Explainable AI (XAI) provides explanations on how those black box decisions of AI systems are made. Such explanations can improve the transparency, persuasiveness and trustworthiness of AI systems and help AI developers to debug and improve model performance. Counterfactual explanation is one of various explanations for AI systems, which are defined as generated examples that are obtained by performing minimal changes in the original instance's features to have a predefined output. Specifically, a causal situation is described in the form “is it input X that caused the model to predict Y” or “Y would not have occurred if X had not occurred.”
(15) For example, consider a person who submitted a credit card application and was rejected by the AI system of a financial company. The applicant may require the company to explain about the decision. The counterfactual explanation not only provides an explanation on why the application was rejected but also helps the applicant to understand what he or she could have done to change this decision, e.g., “if your income was higher than $5000/year, your application would have been approved.” Unlike the explanations based on feature importance that are adopted by some existing systems, counterfactual explanations allow users to explore “what-if” scenarios and obtain a deeper understanding about the underlying AI models.
(16) Existing approaches for counterfactual explanations are generally optimization-based, mostly assuming that the underlying AI model is differentiable and static, i.e., the model is fixed and easy to compute gradients with respect to its inputs. This assumption may be valid for neural network-based models but is largely invalid for tree boosting models such as the widely-applied model XGBoost. Besides, these methods do not handle categorical features with many different values (e.g., non-numerical values) well. For example, an existing method converts categorical features into one-hot encoding representations and then treats them as continuous features. This relaxation introduces many new variables, which complicates the original optimization problem and downgrades the sparsity of the generated counterfactual examples. To ensure real-world feasibility of counterfactual examples, additional constraints have to be imposed on features especially for continuous features, much enhancing the complexity of the original optimization problem.
(17) Thus, in view of the issues with the existing counterfactual explanation systems, embodiments described herein provide a two-stage model-agnostic approach for generating counterfactual explanation via counterfactual feature selection and counterfactual feature optimization. Specifically, counterfactual feature selection is adopted for roughly finding feature columns and values in a query instance so that modifying them can probably change its predicted label. The features may be identified via a nearest-neighbor (NN) search in the feature space of each feature of a data example. Or alternatively, a feature recommendation model is developed for counterfactual feature selection, which can shrink the search space for finding counterfactual examples and reduce the computational time for the next step, i.e., counterfactual feature optimization.
(18) Counterfactual feature optimization is then adopted as a finetuning procedure given the candidate counterfactual features extracted from counterfactual feature selection. The counterfactual feature optimization may be treated as a reinforcement learning (RL) problem that for finding the best counterfactual features. Or alternatively, a gradientless descent (GLD) method can be adopted to further finetune continuous counterfactual features.
(19) Therefore, in this way, the obtained optimal counterfactual features may be used as the explanation that provides more details for users to explain the reasons behind a machine learning decision. In addition, the optimal policy obtained by the RL-based method may provide more details for users to explore “what-if” scenarios and discover if there exists bias in a dataset.

(20) As used herein, the term “network” may comprise any hardware or software-based framework that includes any artificial intelligence network or system, neural network or system and/or any training or learning models implemented thereon or therewith.
(21) As used herein, the term “module” may comprise hardware or software-based framework that performs one or more functions. In some embodiments, the module may be implemented on one or more neural networks.
Overview
(22) FIG. **1** shows a simplified diagram **100** illustrating an overview of generating counterfactual explanations for a machine learning model, according to the embodiments described herein. Diagram **100** shows a framework that may be implemented together with a trained machine learning model **110***f* ( ) e.g., XGBoost or deep learning models. In response to a query **101**, represented by x∈custom character.sup.d, the trained machine learning model **110** may generate a predicted label **106** y. The objective is to generate a counterfactual example **112** x′∈ custom character.sup.d such that its predicted label y′ from the machine learning model f ( ) **110** is different from predicted label **106** y.
(23) For example, the machine learning model **110** may be a binary-class classification problem of predicting whether the income of a person exceeds $50K/yr based on her attributes such as age, education, working hours, and/or the like. The classifier is trained with the census income data. From basic statistical analysis, if her age is above 30 or her education is higher than a bachelor's degree or she has a longer working hours, she probably has an income exceeding $50K/yr. Given a query instance **102** with “age=20, education=high school, working hours=30, predicted income <$50K/yr” to explain, the counterfactual example generation **120** may generate a counterfactual example **112** of “age=30, education=master, working hours=30, predicted income >$50K/yr” with the different predicted label “income >$50K/yr.” Thus, by comparing the attributes between the query instance **102** and counterfactual example **112**, an explanation may be offered that attributes “age=20” and “education=high school” may be determining attributes that cause the prediction “income <$50K/yr.”
(24) FIG. **2** provides an overview of the counterfactual example generation module **120** shown in FIG. **1**, according to embodiments described herein. Traditionally, counterfactual example generation has been formulated as an optimization problem by minimizing the distance between x and x′ subject to that f (x′) y. This optimization problem is usually hard to solve due to the complexity of machine learning model f (.Math.), categorical features in x and the real-world feasibility constraints on x′. The counterfactual example generation module **120** adopts a two-stage approach including the counter feature selection **131** and counterfactual feature optimization **132** as further described below.
(25) Given a query instance **102**, counterfactual feature selection module **131** selects a subset of feature columns and values that can potentially change the prediction. In the example of the binary classifier to predict whether a person's income exceeds $50K/yr given their age, education and working hours, instead of considering the whole domains for age, education and working hours when generating counterfactual examples, the counterfactual feature selection module **131** narrows down the domains to “age ≥30, education∈[bachelor, master, doctorate], working hours ≥40”. The narrowed domains can then be used to simplify the optimization procedure and lead to more reliable explanations since the search space is reduced, and the narrowed domain has already taken into consideration real-world feasibility of possible counterfactual examples.
(26) In addition to narrowing down the domains, as a rough estimate for continuous features in a counterfactual example x′ is often noted in a counterfactual example, e.g., “working hours=range [45,50]” in x′ is accept-able, there is no need to compute an exact value for the attribute such as “working hours=48.5”. Therefore, instead of converting categorical features into continuous ones by traditional optimization approaches, continuous features are converted into categorical ones by discretizing them into k bins before counterfactual feature selection.

(27) Thus, as shown in FIG. **2**, counterfactual feature selection module **131** picks a subset **204** of feature columns and discrete values, such as column "age" and values "30," "35," "40," column "education" and values "Bachelor," "Master," "PhD," etc. Further details of the algorithms of feature selection performed by the counterfactual feature selection module **131** are described below in relation to FIGS. **5-7**. For example, the counterfactual feature selection may employ a nearest neighbor search as described in Alg. 1 in FIG. **5**.
(28) Next, given the query instance **102** and its predicted label y (income <$50K/yr) and the selected counterfactual features **204**, the counterfactual feature optimization module **132** determines which features in **204** are to be manipulated and what values to be set so that the predicted label of the modified instance x′ is different from y.
(29) Counterfactual feature optimization then determines the best feature value for each selected feature column, e.g., "Age=30, Education=Master, Working hours=45," etc. A batch of counterfactual examples **206** can then be constructed based on the optimized feature values. Further details of the algorithms of feature optimization performed by the counterfactual feature optimization module **131** are described below in relation to FIGS. **9-10**. For example, the counterfactual feature optimization **132** may employ Alg. 4 in FIG. **9**.
(30) Given a set of counterfactual examples **206** generated from the counterfactual feature optimization **132**, the counterfactual example selection **133** is configured to pick a set of diverse counterfactual examples such that the proximity of each counterfactual example is as high as possible (or the distance between each counterfactual example and the original query is as small as possible). Generating diverse counterfactual examples may help to get better understanding of machine learning prediction. The counterfactual example selection **133** may be implemented by Alg. 6 in FIG. **11**, which includes a greedy algorithm to generate top k counterfactual examples based on proximity and diversity.
(31) The continuous feature fine-tuning module **134** is then applied if one wants to optimize continuous proximity further to generate more actionable explanations. Specifically, one concern about discretizing continuous features is that the generated counterfactual examples may have better or more actionable values for those continuous features if they are optimized directly instead of being discretized via ordinal encoding. Therefore, the continuous feature fine-tuning module **134** may further optimize continuous counterfactual features to improve proximity. In one implementation, the continuous feature fine-tuning module **134** may implement Alg. 5 in FIG. **10**B, e.g., a gradient-less descent (GLD) method for refining continuous counterfactual features.
(32) Therefore, after continuous feature fine-tuning, the final counterfactual examples **208** may be generated, e.g., "age=25, work class=private, education=bachelor, work hours=42.5", etc.
(33) Computer Environment
(34) FIG. **3** is a simplified diagram of a computing device for generating counterfactual explanations, according to some embodiments. As shown in FIG. **3**, computing device **300** includes a processor **310** coupled to memory **320**. Operation of computing device **300** is controlled by processor **310**. And although computing device **300** is shown with only one processor **310**, it is understood that processor **310** may be representative of one or more central processing units, multi-core processors, microprocessors, microcontrollers, digital signal processors, field programmable gate arrays (FPGAs), application specific integrated circuits (ASICs), graphics processing units (GPUs) and/or the like in computing device **300**. Computing device **300** may be implemented as a stand-alone subsystem, as a board added to a computing device, and/or as a virtual machine.
(35) Memory **320** may be used to store software executed by computing device **300** and/or one or more data structures used during operation of computing device **300**. Memory **320** may include one or more types of machine readable media. Some common forms of machine readable media may include floppy disk, flexible disk, hard disk, magnetic tape, any other magnetic medium, CD-ROM, any other optical medium, punch cards, paper tape, any other physical medium with patterns of holes, RAM, PROM, EPROM, FLASH-EPROM, any other memory chip or cartridge, and/or any

other medium from which a processor or computer is adapted to read.
(36) Processor **310** and/or memory **320** may be arranged in any suitable physical arrangement. In some embodiments, processor **310** and/or memory **320** may be implemented on a same board, in a same package (e.g., system-in-package), on a same chip (e.g., system-on-chip), and/or the like. In some embodiments, processor **310** and/or memory **320** may include distributed, virtualized, and/or containerized computing resources. Consistent with such embodiments, processor **310** and/or memory **320** may be located in one or more data centers and/or cloud computing facilities.
(37) In some examples, memory **320** may include non-transitory, tangible, machine readable media that includes executable code that when run by one or more processors (e.g., processor **310**) may cause the one or more processors to perform the methods described in further detail herein. For example, as shown, memory **320** includes instructions for a counterfactual explanation module **330** that may be used to implement and/or emulate the systems and models, and/or to implement any of the methods described further herein. In some examples, the counterfactual explanation module **330**, may receive an input **340**, e.g., such as query instances, via a data interface **315**. The data interface **315** may be any of a user interface that receives user query, or a communication interface that may receive or retrieve a previously stored query sample from the database. The counterfactual explanation module **330** may generate an output **350** such as counterfactual examples.
(38) In some embodiments, the counterfactual explanation module **330** may further includes the counterfactual feature selection module **131**, the counterfactual feature optimization module **132**, the counterfactual example selection module **133** and the continuous feature fine-tuning module **134**, which perform similar operations as described in relation to FIG. **2**. In some examples, the counterfactual explanation module **330** and the sub-modules **131-134** may be implemented using hardware, software, and/or a combination of hardware and software. For instance, the counterfactual feature selection module **131** may include a structure shown at **800** in FIG. **8** and implement the algorithms and/or processes described in relation to FIG. **6-7**. The counterfactual feature optimization **132** may implement the algorithms and/or processes described in relation to FIG. **9-10**. The counterfactual example selection module **133** may implement the algorithm and/or process described in relation to FIG. **11**. The continuous fine-tuning module **134** may implement the algorithm and/or process described in relation to FIG. **10**B.
Example Embodiments of Counterfactual Example Generation
(39) FIG. **4** provides an example logic flow diagram illustrating a method of counterfactual example generation, according to embodiments described herein. One or more of the processes **402-414** of method **400** may be implemented, at least in part, in the form of executable code stored on non-transitory, tangible, machine-readable media that when run by one or more processors may cause the one or more processors to perform one or more of the processes **402-414**. In some embodiments, method **400** may correspond to the method used by the module **330**.
(40) At process **402**, a query instance (e.g., **102** in FIG. **2**) may be received, e.g., via the data interface **315** in FIG. **3**. The query instance may include a plurality of feature columns (e.g., "age," "working hours," "education," etc.) and corresponding feature values (e.g., age="20," working hours="30," etc.).
(41) At process **404**, a machine learning model (e.g., **110** in FIG. **1**) may generate a predicted label in response to the query instance. For example, a predicted binary label of "not >$50K/yr" of predicted income level for the input query instance.
(42) At process **406**, a subset of feature columns, each associated with a respective subset of feature values may be identified such that the subset of feature columns and values may potentially lead to a different predicted label. For example, process **406** may be implemented by the counterfactual feature selection module **131** based on K-nearest neighbor search (as further described in relation to FIG. **5**) or a recommendation model (as further described in relation to FIGS. **6-8**).
(43) At process **408**, given the selected feature columns and feature values from process **406**, optimal features in the query instance are determined to be manipulated and values are set for these

features from the identified subset of feature columns and values. In this way, counterfactual examples are constructed based on the determined optimal features and values at process **410**. Processes **408**-**410** may be performed by the counterfactual feature optimization module **132** using a reinforcement learning method, as further described in FIG. **9**, or alternatively a gradient-less descent method, as further described in FIG. **10**A. Counterfactual examples may be generated without the gradients of the machine learning model and thus may be applied to all kinds of classification models.

(44) At process **412**, the given the optimized counterfactual examples, a subset of counterfactual examples may be selected to provide proximity and diversity. For example, the selection may be performed via Alg. 6 in FIG. **11**. The selected diverse counterfactual examples are then fine-tuned for continuous features, which may be performed via the GLD algorithm in FIG. **10**B.

(45) At process **414**, the generated counterexample can be outputted with the predicted label as an explanation why the predicted label is generated in response to the specific query instance.

(46) FIG. **5** provides an example pseudo-code segment illustrating an algorithm of the nearest neighbor search approach for generating counterfactual features, according to embodiments described herein. The data for training the machine learning model **110**, e.g., a binary classifier, is divided into two classes according to the labels. At step (1) of Alg. 1, for each class label l, a search index, tree.sub.l, is built for fast nearest neighbor search with the divided dataset associated with the label l, e.g., {x.sub.i|y.sub.i=l for (x.sub.i, y.sub.i)∈custom character}. The training dataset custom character can be either a subset of the examples or the whole examples for training the machine learning model **110**.

(47) At step (2) of Alg. 1, the counterfactual feature selection module **131** finds the K nearest neighbors of x in the different class label y′=1-y, using the search index tree.sub.y′. Specifically, the distance metric applied here is the Euclidean distance. Note that the continuous features in the query instance x have been discretized into bins. One-hot encoding is applied for categorical features, while applying ordinal encoding for continuous features so that information of the closeness can be preserved, e.g., age of 20-25 is closer to 25-30 than 45-50.

(48) At steps (3)-(6) of Alg. 1, the frequency of the changed feature columns and the frequency of the changed feature values in the K nearest neighbors compared to the original query instance x are computed.

(49) At step (7) of Alg. 1, the counterfactual feature selection module **131** sorts the feature columns in descending order with respect to the computed frequency of the changed feature columns. Module **131** then selects the top s feature columns.

(50) At step (8) of Alg. 1, module **131** sorts the elements in the changed values in descending order with respect to the computed frequency of the changed feature values. Module **131** then selects the top m values for each feature column.

(51) At step (9) of Alg. 1, the selected top s feature columns, denoted by custom character, and the selected top m feature values, denoted by custom character(custom character), are output by module **131** as the selected counterfactual features.

(52) Alg. 1 can be implemented efficiently to generate at most s×m candidate features. Alg. 1 also allows users to specify actionable feature columns or the columns they care about most. In this way, before constructing the search index, instead of considering all the features in x, module **131** only concerns a subset of the features specified the users and builds a search index based on this subset, which largely reduces computational cost.

(53) FIG. **6** provides an example pseudo-code segment illustrating an algorithm of building a recommendation model for generating counterfactual features, according to embodiments described herein. Alg. 2 shown in FIG. **6** illustrates an alternative approach of generating counterfactual features based on a recommendation model. Specifically, as the objective of module **131** is to choose several features that potentially change the predicted label, this problem can be rephrased as a recommendation problem, i.e., given an input x with d different features and predicted label y, the

recommendation model is trained to recommend: (1) k feature columns such that varying the values in these features can potentially change the predicted label from y to y′=1-y; and (2) m candidate values for each feature selected in the previous step such that setting to these values can potentially flip the predicted label.

(54) Thus, given the machine learning model **110**, e.g., a classifier f (.Math.), and the training data custom character another machine learning model g(.Math.) may be trained to perform this recommendation task. Alg. 2 shows an example method of construct a training dataset for the counterfactual feature recommendation task in order to design the machine learning model g ( ) for it with high recommendation accuracy.

(55) At step (1) of Alg. 2, for each query instance x.sub.i∈custom character, a predicted label f(x.sub.i) is generated to result in set X={(x.sub.i, f(x.sub.i)) for i={1, . . . , n}. As the counterfactual explanation is based on the trained classifier f (.Math.), the data used to build the training dataset is the classifier's prediction results X.

(56) At step (2) of Alg. 2, for each class label l, a search index, tree.sub.l, is built for fast nearest neighbor search with the divided dataset associated with the label l, e.g., {*x*.sub.i|*y*.sub.i=*l* for (*x*.sub.i,*y*.sub.i)∈*X}*.

(57) At steps (3)-(4) of Alg. 2, for each instance (x.sub.i, y.sub.i) in X, the K nearest neighbors, denoted as set Z of xi in the class with label 1-y.sub.i are found, e.g., via the search index tree.sub.1-yi.

(58) At step (5) of Alg. 2, from Z, the feature columns c and feature values v that are different from those in xi into the dataset R, which is the training dataset for the recommendation model.

(59) Thus, for (x, y, c, v)∈R, the goal of the recommendation model is to predict feature column c and feature value v given instance x and its predicted label y. Note that there may exist some duplicate instances in R, i.e., the same (x, y, c, v)∈R may appear multiple times. The duplicate instances do not need to be removed since if an instance (x, y, c, v) appear repeatedly, it is probable that (c, v) are candidate features for x, which is equivalent to impose more weights on this instance during training the recommendation model.

(60) FIG. **7** provides an example pseudo-code segment illustrating an algorithm of generating counterfactual feature selections based on a recommendation model, according to embodiments described herein. Given the training dataset R generated by Alg. 2 in FIG. **6**, assume there are r instances in R, i.e., R={(x.sub.i, y.sub.i, c.sub.i, v.sub.i)|.sub.i=1.sup.r}.

(61) At step (1) of Alg. (3), the recommendation model takes (x.sub.i, y.sub.i) as the inputs and generates: 1) the probability P∈custom character.sup.d for feature column selection, e.g., a feature column with a higher probability means that changing its value may more likely flip the predicted label, and 2) the probabilities {Q(c)∈custom character.sup.nc|.sub.c=1.sup.d} for feature value selection for each feature column where n.sub.c is the number of the candidate values for feature column c, e.g., for feature column c, a feature value v with a higher probability indicated by Q(c) means that setting the value of c to v may more likely flip the predicted label.

(62) At step (2) of Alg. 3, the probabilities P are sorted in descending order, and the top s feature columns with the highest probabilities are selected, denoted by set C.

(63) At step (3) of Alg. 3, for each column c∈C, the probabilities Q(c) are sorted in descending order and the top m feature values are selected, denoted by V(c).

(64) At step (4) of Alg. 3, the set of feature columns C and corresponding set of feature values V(C) are returned as the selected counterfactual features.

(65) Compared with Alg. 1 in FIG. **5**, Alg. 3 only generates s×m candidate features instead of d×m in the worst case. When s is much smaller than d, Alg. 3 may significantly reduce the search space for finding counterfactual examples.

(66) FIG. **8** shows a simplified diagram **800** illustrating the neural network model structure of a recommendation model for feature selection, according to embodiments described herein. Diagram

800 shows that any continuous features in query instance x are discretized into xi, . . . , xd before feeding into the network. For example, each feature and the predicted label **802***a*-*n* are converted into a low-dimensional and dense real-valued embedding vectors **804***a*-*n*. Then the two embeddings **804***a*-*n* are added together and followed by layer normalization **806***a*-*n*, respectively. To model the correlation between different features, a transformer layer comprising a self-attention sublayer **808**, a feed-forward sublayer **810** and a normalization layer **812**.

(67) The output hidden states of the transformer layer are then concatenated at module **814** together and then fed into two multi-layer perceptrons (MLPs) **815***a*-*b*, i.e., one for feature column selection and the other one for feature value selection. Each MLP **815***a* or **815***b* then outputs to its respective softmax module(s) to generate the respective probabilities, P or Q(c), as described in Alg. 3 in FIG. **7**.

(68) In one embodiment, the model shown at diagram **800** may have multiple transformer layers for modeling hidden feature representations.

(69) FIG. **9** provides an example pseudo-code segment illustrating an algorithm of RL-based counterfactual feature optimization for generating a counterfactual example given selected counterfactual features, according to embodiments described herein. The counterfactual feature optimization problem can be formulated as a reinforcement learning problem. Specifically, suppose that C={c.sub.1, .Math., c.sub.s} and V(c.sub.i) {v.sub.1, .Math., v.sub.m} for i=1, .Math., s. The reinforcement learning environment is defined by the classifier f(.Math.), the instance (x, y), the candidate features C and V(C). The action is defined by the feature columns and values to be modified in order to construct the counterfactual example x′. The state is x′ by taking an action on x. The reward function is the prediction score of label y′=1-y. For each round t, an action at is taken to construct x.sub.t′ and then the environment sends the feedback reward r.sub.t=f.sub.1-u (x.sub.t′). Let π.sub.θ(.Math.) be a stochastic policy for selecting actions parameterized by θ. Then the objective is to find the optimal policy that maximizes the cumulative reward:

(70) $\max J(\quad) := \mathbb{E}\quad[\text{.Math.}_{t=1}^{T} r_t]$, (1) $s.t.$ ofselectfeaturesby $\leq w$

where w denotes a sparsity constraint on the policy π.sub.θ such that the number of the selected feature columns to modify cannot be larger than constant w.

(71) The policy TEO may be formulated as follows. Let μ=(μ.sub.1, .Math., μ.sub.s)∈{0, 1}.sup.s be a random vector whose ith element indicates whether the ith feature column is selected or not, e.g., μ.sub.i=1 means it is selected, and P be the probability distribution of μ. For each c∈C, let v.sub.c∈{1, .Math., m} be a random variable indicating which feature value is selected for the cth feature, v be the random vector (v1, . . . , vc) and Qc be the probability distribution of v.sub.c. Conditioned on μ, v.sub.1, .Math., v.sub.c are independent from each other. Now that constructing counterfactual example x′ involves two steps, i.e., one step for selecting features to modify and the other step for picking the value for the selected features. Thus, the step to select the value of feature c only depends on c without depending on other features.

(72) An action can be represented by μ and v, e.g., a={(μ.sub.1=1, v.sub.1=2), (μ.sub.2=0, v.sub.2=None), (μ.sub.3=1, v.sub.3=1)}, meaning that the first and third features are set to values 2 and 1, respectively. Then policy π.sub.θ can be defined by:

πθ(*a*)=custom character(μ)Π.sub.c=1.sup.s*Q*.sub.c(*v*.sub.c).sup.1[μ.sup.c.sup.=1] (2)

where 1[.Math.] is the indicator function which equals 1 if the condition is met or 0 otherwise. P is constructed by s independent Bernoulli distribution parameterized by p.sub.c and Qc is defined by the softmax function parameterized by q c∈custom character.sup.m, namely:

(73) $(\quad) = \text{.Math.}_{c=1}^{s}\ \text{Bernoulli}(p_c, \quad_c), Q_c(v) = \frac{e^{q_{c,v}}}{\text{.Math.}_{i=1}^{m} e^{q_{c,i}}}$ (3)

where q.sub.c,i is the ith element of q.sub.c, then θ=(p.sub.1, .Math., p.sub.s, q.sub.1, . . . , q.sub.s). Under this assumption, the sparsity constraint w is equivalent to ‖p‖.sub.0≤w where p=(p.sub.1, .Math., p.sub.s) and ‖.Math.‖.sub.0 is the number of nonzero elements. Therefore, the optimization

problem (1) can be reformulated as:

(74) $\max \mathbb{E}\ [\ .\text{Math.}\sum_{t=1}^{T} r_t], s.t.\ .\text{Math. } p\ .\text{Math.}_0 \le w$ (4)

where the policy π.sub.θ is defined by (2).

(75) As the optimization problem with l.sub.0-norm constraint may be difficult to compute, a relaxed form of the optimization problem may be obtained:

(76) $\min\text{-}\mathbb{E}\ [\ .\text{Math.}\sum_{t=1}^{T} r_t] + \ _1\ .\text{Math. } p\ .\text{Math.}_1 + \ _2\ .\text{Math.}\sum_{c=1}^{s} p_c \log p_c$ (5)

where λ.sub.1 and λ.sub.2 are constants. The first regularization term ‖p‖.sub.1 is used to encourage sparse solutions and the second regularization term is the probability entropy that encourages doing exploration during minimization via policy gradient.

(77) In step (1) of Alg. 4 shown in FIG. **9**, the relaxed problem shown in Eq. (5) is solved by finding the optimal policy π.sub.θ*, e.g., θ*=(p*.sub.1, .Math., p*.sub.s, q*.sub.1, .Math., q*.sub.s). via the REINFORCE algorithm. Further details of the REINFORCE algorithm can be found in [33, 36], which are hereby expressly incorporated by reference herein in their entirety.

(78) At step (2) of Alg. 4, the obtained (p*.sub.1, .Math., p*.sub.s) are sorted in descending order. The top w features with the highest values of p* may be selected, where w is the upper bound of sparsity. The set of selected features is denoted by C*.

(79) At step (3) of Alg. 4, for each selected feature c in C*, the top feature value v.sub.c* with the highest value of q.sub.c is selected.

(80) At step (4) of Alg. 4, the set of the selected features and the selected feature values custom character*={(c,v.sub.c*)|c∈C*} is sorted in descending order according to the corresponding p.sub.c*.

(81) At step (5) of Alg. 4, a greedy method is applied to construct counterfactual examples. For example, for every (c, v) in the set sorted set custom character*, set x′[c]=v, and if f.sub.1-y (x′)>0.5, the counterfactual example x′ and the optimal features custom character* are returned.

(82) At steps (6)-(7) of Alg. 4, actions are sampled based on the optimal policy π.sub.θ*, and a batch of counterfactual examples are generated. The generated counterfactual examples are thus provided.

(83) In this way, in addition to the generated counterfactual examples x′, the selected features (and values) set custom character* can also be utilized for providing an explanation that provides the counterfactual feature values for each feature in C*.

(84) In addition, if the definitions of custom character and Q.sub.c in Eq. (3) is generalized as follows:

custom character(μ)=Π.sub.c=1.sup.s1[*p*.sub.c=μ.sub.c*]*,Q*.sub.c(*v*)=1[*q*.sub.c,v=1] (6)

(85) Given the instance (x, y) to explain and the domains defined by C and V(C) generated from counterfactual feature selection, the counterfactual example x* constructed by the optimal policy from Eq. (4) with π.sub.θ defined by Eq. (2) and Eq. (6) is also an optimal solution of the following optimization problem:

(86) $\max_{x'} f_{1-y}(x'), s.t.\ .\text{Math. } x' - x\ .\text{Math.}_0 \le w$ (7)

(87) Therefore, while optimizing Eq. (4) with custom character and custom character.sub.c defined by Eq. (6) can be NP hard, a relaxation by replacing Eq. (6) with its “soft” formulation Eq. (3) can be done. This means that Alg. 4 that solves Eq. (5) actually approximately solves Eq. (7).

(88) FIG. **10**A provides an example pseudo-code segment illustrating an algorithm of GLD-based counterfactual feature optimization for generating a counterfactual example given selected counterfactual features, according to embodiments described herein. A relaxation of the optimization problem (7) by moving the constraint into the objective function is given as follows:

(89) $\min_{x'}\text{-}\ f_{1-y}(x') + \frac{1}{d}\ .\text{Math. } x' - x\ .\text{Math.}_0$ (8)

where x is the query instance, y is its corresponding label, d is the number of the features and λ>0 is the regularization weight. Thus, the counterfactual feature optimization module **132** is configured to find the counterfactual example x′ which minimizes Eq. (8).

(90) Thus, given C={c.sub.1, .Math., c.sub.s} and V(c.sub.i)={v.sub.1, .Math., v.sub.m} for i=1, .Math., s selected from the previous step, define μ∈custom character.sub.s and v.sub.i∈ custom character.sup.m for i=1, .Math., s such that x′=g(x, μ, v.sub.1, .Math., v.sub.s) where g satisfies

(91) $x'[c_i] = \begin{cases} x'[c_i] & \text{if } [i] \le 0.5; \\ v_j \text{ where } j = \operatorname{argmax}_k v_i[k] & \text{otherwise}. \end{cases}$

(92) In other words, x′ is constructed by the values of {μ, v1, . . . , vs}, namely, for each index i=1, . . . , s and the corresponding feature column c.sub.i∈C, μ[i]<=0.5 means “without modifying this feature”, i.e., x′[ci] is set to x[c.sub.i], while μ[i]>0.5 means “modifying this feature”, i.e., x′ [c.sub.i] is set to v.sub.j∈V(c.sub.i) whose index j has the highest value among the values in v.sub.i. Then Problem (8) can be reformulated as

(93) $\min_{, v_1, \text{ .Math. }, v_s} - \quad f_{1-y}(g(x, \quad, v_1, \text{ .Math. }, v_s)) + \frac{1}{d} \text{ .Math. } 1[\quad > 0.5] \text{ .Math. }_0 . \quad (9)$

(94) Alg. 5 in FIG. **10** solves the optimization problem (9). Specifically, at step (1) of Alg. 5, an inner loop limit K is set to be K=log (R/r) where R denotes the maximum search radius, and r denotes the minimum search radius.

(95) At steps (4)-(8) of Alg. 5, the loop **1001** computes the set custom character by adding computed (x′.sub.k,μ.sub.k,v.sub.1,k, . . . ,v.sub.s,k) if f.sub.1-y(x′.sub.k)>0.5 for k=1, . . . , K. At the current time step t, the resulting x′, μ, v.sub.1, . . . , v.sub.s is stored be the best solution of Eq. (9) in custom character.

(96) At steps (3)-(10) of Alg. 5, the loop **1002** repeats the loop **1001** for all time steps t=1, . . . , T. More sparse solutions are obtained, i.e., if μ[i] switches on (μ[i]>0.5), μ[i] is reset to a number slightly larger than 0.5, e.g., 0.55, making that it has a higher chance to switch off (μ[i] <=0.5) in the next loop.

(97) At steps (2)-(11) of Alg. 5, the loop **1003** repeats the loop **1002** for all training epochs. Then the counterfactual example x′ is selected as the one that minimizes Eq. (8).

(98) The RL-based method (Alg. 4 in FIG. **9**) and GLD method (Alg. 5 in FIG. **10**) may also be applied to explain multivariate time series classification. Specifically, given a query instance x∈ custom character.sup.d×t (d time series of length t), a classifier f (.Math.) and a desired label y′, the goal is to find a counterfactual example x′ constructed by the substitutions of entire time series from a distractor sample {tilde over (x)} chosen from the training dataset to x, and to minimize the number of substitutions, i.e., min.sub.C−λf.sub.y′(x′)+|C|/d, where x′[i, :] equals {tilde over (x)}[i, :] if i∈C or x[i, :] otherwise. The distractor sample {tilde over (x)} may be found by the nearest neighbor search method discussed at Alg. 1 of FIG. **5**, with number of nearest neighbors K=1 and number of selected values m=1. Then for determining which time series to be substituted, RL-based method or GLD method may be applied by only considering feature column selection, i.e., optimizing μ, because there is only one candidate value for each feature.

(99) Alternatively, FIG. **10**B provides an example pseudo-code segment illustrating the GLD method for continuous feature fine-tuning at module **133**, according to embodiments described herein.

(100) Given a counterfactual example x′ generated by the previous stage, let C be the set of the continuous features such that x′[i]≠x[i] for each i∈C, i.e., the set of the continuous features that are modified to construct x′. Without loss of generality, assume that the value of each continuous feature lies in [0, 1]. The goal is to optimize the values x′[i] for i∈C such that the new counterfactual example {circumflex over (x)} satisfies f.sub.1-y ({circumflex over (x)})>0.5 d the

difference between {circumflex over (x)} and x on the continuous features, i.e. Σ.sub.i∈C|{circumflex over (x)}[i]−x[i]|, is as small as possible. Thus another form of the Eq. (8) can be represented as:

(101) $\min_{z} \frac{1}{\text{.Math. } C \text{ .Math.}} \text{.Math.} \sum_{i \in C} \text{.Math. } z[i] - x[i] \text{ .Math. ,}$

$s.t.\, f_{1-y}(z) > 0.5, z[i] = x'[i] \forall i \text{ .Math. } C, z[i] \in [0,1], \forall i \in C.$

(102) This alternate form of Eq. (8) can be approximately solved via Alg. 5 in FIG. **10**B, which is an adapted form of FIG. **10**A for continuous feature fine-tuning.

(103) FIG. **11** provides an example pseudo-code segment illustrating an algorithm of generating diverse counterfactual examples for counterfactual example selection, according to embodiments described herein. Diverse counterfactual examples may help to get better understanding of machine learning prediction. In one implementation, Alg. 6 shown in FIG. **11** provides a simplified algorithm for generating diverse counterfactual examples with the RL-based (Alg. 4 in FIG. **9**) and GLD (Alg. 5 in FIG. **10**) methods.

(104) In another implementation, Alg. 6 may be applied for selecting counterfactual examples by module **133**.

(105) At step (1a) or (1b) of Alg. 6, a set of counterfactual examples E is constructed. For example, for the RL-based method shown in Alg. 4 in FIG. **9**, set E may be constructed by the optimal policy, i.e., by sampling an action via the optimal policy, constructing the corresponding counterfactual example x′, and adding x′ into ε if f (x′) >0.5. For the GLD method shown in Alg. 5 in FIG. **10**, set ε is the union of custom character constructed at each step t.

(106) At step (2) of Alg. 6, the examples in set E are sorted in ascending order based on the value −λf.sub.1-y (x′)+‖x′−x‖.sub.0/d, i.e., the objective function in Eq. (8). This function makes a trade-off between the sparsity and the prediction score for the desired label 1-y.

(107) In another example, the examples in set ε are sorted in descending order based on the sum of categorical proximity and continuous proximity, e.g.,
Proximity=−‖*x*′.sub.cat−*x*.sub.cat‖.sub.0−‖(*x*′.sub.con−*x*.sub.con)/*m*.sub.con‖.sub.1,
where x.sub.cat and x.sub.con are the categorical and continuous features respectively, and m.sub.con is the media of x.sub.con over the training data.

(108) At steps (3)-(5) of Alg. 6, “duplicate” examples from the sorted set are removed in a greedy manner to make the counterfactual feature columns (their feature values are different from x) of the selected examples less overlapped with each other. Specifically, at step (4), for each example x′ in the sorted set, let D(x′) be the set of the features in x′ that are different from x. At step (5), if there exists any feature in D(x′) also belonging to more than K sets D(z) where z is ranked higher than x′ in the sorted set, then example x′ is skipped. Otherwise, if no more than K sets D(z) where z is ranked higher than x′ in the sorted set, x′ is added to the diverse set.

(109) Thus, at step (6) of Alg. 6, after each x′ in the sorted set has been iterated, the diverse set is returned.

Example Performance

(110) FIGS. **12-27** provide example data tables illustrating data experiment results of the counterfactual example generation, according to one embodiment. Five tabular datasets and one time series classification dataset are used for running data experiments to evaluate the counterfactual example generation module described herein.

(111) Dataset Adult Income is used to predict whether income exceeds $50K/yr based on census income data and is available on the UCI machine learning repository. 8 features, namely, age, working hours, work class, education, marital status, occupation, race and gender are extracted. After data cleaning, the dataset has 32561 instances.

(112) Breast Cancer dataset is the “Breast Cancer Wisconsin (Original) Dataset” described in Mangasarian et al., Breast Cancer Diagnosis and Prognosis Via Linear Programming. Operations Research 43, 4 (1995), pp. 570-577, 1995. There are 699 instances and each instance has 9 features,

e.g., clump thickness, uniformity of cell size, uniformity of cell shape. The task is to classify whether an instance is malignant or benign.
(113) COMPAS dataset contains 6172 instances. 6 features, i.e., number of priors, score factor, misdemeanor, gender, age and race are extracted. The task is to predict which of the bail applicants will recidivate in the next two years.
(114) Australian Credit dataset concerns credit card applications, which has 690 instances, of which each consists of 14 features (6 continuous features and 8 categorical features) and 1 class label that quantifies the approval decision. The task is to predict if the credit application was approved or rejected to a particular customer.
(115) Titanic dataset is used to predict which passengers survived the Titanic shipwreck. It has 891 instances after data cleaning. 7 features, namely, age, fare, ticket class, sex, sibsp (number of siblings/spouses aboard), parch (number of parents/children aboard), embarked are extracted.
(116) Taxonomist dataset contains runs of 11 different applications with various input sets and configurations. The goal is to classify the different applications. It has 4728 samples and each sample has 563 time series.
(117) Each dataset is divided into 80% training and 20% test data for evaluating machine learning classifiers. Three popularly used classifiers, an XGBoost model and a MLP model for tabular classification, and a random forest model for time series classification. The categorical features are converted into one-hot encoding and the continuous features are scaled between 0 and 1. The hidden layer sizes and the activation function for MLP is ReLU, respectively. The max depth for XGBoost is 8. The number of estimators for random forest is 20.
(118) For Algorithm 1, the number of the nearest neighbors K is set to 30, the number of selected feature columns s is set to 8 and the number of the selected values for each feature m is set to 3. Counterfactual feature optimization is performed via the REINFORCE algorithm. ADAM (Kingma et al., Adam: A Method for Stochastic Optimization, arXiv:cs.LG/1412.6980, 2017) is used as the optimizer with learning rate 0.1, batch size 40 and 15 epochs. In order to make the REINFORCE algorithm stable, the reward function used here is r=f.sub.1-y (x′) −b where b is the median value of f.sub.1-y (.Math.) in one batch instead of r=f.sub.1-y (x′) for variance reduction. The regularization weights λ.sub.1 and λ.sub.2 are set to 2. For RL-method, the sparsity parameter w is set to 6. For GLD method, E=3, T=20, R=0.25, r=0.0005 and A=0.1, which works well for all the experiments.
(119) The following approaches are used as baseline for generating counterfactual examples in the experiments:
(120) Greedy denotes the sequential greedy method proposed in Ates et al., Counter-factual Explanations for Machine Learning on Multivariate Time Series Data, arXiv:cs.LG/2008.10781. Given the candidate counterfactual features selected from nearest neighbor search (Algorithm 1), the greedy method modifies the features in x sequentially until the predicted label flips.
(121) A SHAP-value based method is proposed in Rathi, Generating Counterfactual and Contrastive Explanations using SHAP. arXiv:cs.LG/1906.09293. It first computes the SHAP values for all the features in x and then constructs counterfactual examples by changing the features whose SHAP values of the desired class y′=1-y are negative.
(122) DiCE method is proposed in Mothilal et al., Explaining Machine Learning Classifiers through Diverse Counterfactual Explanations. In Proceedings of the 2020 Conference on Fairness, Accountability, and Transparency, Association for Computing Machinery, pp. 607-61′7, and Rathi 2019.
(123) CERTIFAI is a custom genetic algorithm to generate counterfactuals proposed by Sharma et al., CERTIFAI: A Common Framework to Provide Explanations and Analyse the Fairness and Robustness of Black-Box Models, in Proceedings of the AAAI/ACM Conference on AI, Ethics, and Society, pp. 166-172,2020. It is a model-agnostic method and can be applied to black-box classification models.

(124) Foil Trees is proposed by Waa et al., Contrastive Explanations with Local Foil Trees, arXiv:stat.ML/1806.07470, 2018, utilizing locally trained one-versus-all decision trees to identify the disjoint set of rules that causes the tree to classify data points as the foil and not as the fact.
(125) The RL and GLD methods described in FIGS. **9-10** are applied. The candidate counterfactual features are generated by Algorithm 1.
(126) To evaluate the methods for generating counterfactual examples, the validity, sparsity and diversity metrics as well as the prediction scores for desired classes. Validity is the fraction of examples returned by a method that are valid counterfactuals, e.g., f.sub.1-y (x′) >0.5. Sparsity indicates the number of changes between the original input x and a generated counterfactual example x′. Diversity measures the fraction of features that are different between any two pair of counterfactual examples (e.g., generate K counterfactuals per query instance). For sparsity, the lower the better. For validity, diversity and prediction scores, the higher the better.
(127) Table 1 in FIG. **12** and Table 2 in FIG. **13** demonstrate the validity and sparsity for Greedy, SHAP-based, CERTIFAI, Foil Trees, the RL and GLD with the XGBoost classifier. DiCE is not included here because it is hard to handle the XGBoost classifier. The RL and GLD methods have the highest validity. RL obtains the lowest sparsity for all the datasets except “Adult” compared with other approaches, and GLD has the lowest sparsity for “Adult”. Note that the validity of Foil Trees is quite low compared with other methods and it fails for “Breast” and “Credit”. Table 3 in FIG. **14** shows that the RL method achieves the highest prediction scores of the desired labels, meaning that the counterfacutals generated by it are closer to the desired classes.
(128) Table 4 in FIG. **15** and Table 5 in FIG. **16** demonstrate the validity and sparsity for Greedy, SHAP-based, CERTIFAI, DiCE, Foil Trees, RL and GLD with the MLP classifier. DiCE, the RL and GLD are able to find valid counterfactual examples for all the test instances, outperforming the other approaches. But in terms of sparsity, DiCE has much larger sparsity than the proposed methods, e.g., 2.93 for “Adult” and 2.10 for “COMPAS”, while RL/GLD achieves 1.47/1.50 for “Adult” and 1.43/1.49 for “COMPAS”. For “Credit” and “Titanic”, Greedy, SHAP-based and RL/GLD have similar performance with the MLP classifier, which is different from that with the XGBoost classifier. Running time per query for all these methods as shown in Table 7 of FIG. **18**. The Foil Trees method is fastest but it obtains the worst performance. DiCE is the slowest method which takes about 3 seconds per query, making it impractical for real applications. RL/GLD methods achieve comparable speed as Greedy and SHAP-based while obtaining better performance in terms of validity, sparsity and prediction scores.
(129) Tables 8 in FIG. **19** and Table 9 in FIG. **20** give several counterfactual examples generated by the counterfactual example generation approach described herein. For the Adult dataset, the counterfactual examples show that studying for an advanced degree or spending more time at work can lead to a higher income. It also shows a less obvious counterfactual example such as getting married for a higher income. This kind of counterfactual example is generated due to correlations in the dataset that married people having a higher income. For the Titanic dataset, the results show that the persons who are female or have higher ticket classes have a bigger chance to survive from the disaster.
(130) Besides the counterfactual examples, the proposed approach can also take counterfactual features extracted from the policy learned by RL method as the explanation to explore “what-if” scenarios, which cannot be easily achieved by other approaches. In Table 8 and Table 9, the top-4 selected feature columns and top-1 feature values are listed for each example. For the first example in Table 8, these counterfactual features “Education.fwdarw.Doctorate, Working hours.fwdarw.37.5, Workclass.fwdarw.Private, Occupation.fwdarw.Professional” are obtained, meaning that he/she will have a higher income if he/she has a higher degree, longer working hours and a more professional occupation. For the first example in Table 9, “Sex.fwdarw.female, Ticket class.fwdarw.1, Embarked.fwdarw.C, SibSp.fwdarw.0” is obtained, which means gender being female or having a higher class or having no siblings/spouses leads to a bigger survival chance.

These counterfactual explanations help detect bias in the dataset which may reflect some bias in our society under certain scenarios. For example, women were more likely to survive because society's values dictate that women and children should be saved first. And rich passengers were more likely to survive because they had rooms on the higher classes of the ship so that they may have the privilege to get to the boats more easily.

(131) Additional experiment computes the average number of the changes for each feature to construct counterfactual example x′ and compare it with the feature importance of the original input x. The feature importance is computed by taking the average of the SHAP values of the samples in the test dataset. FIG. **21** shows this comparison in the Adult dataset where features 0 to 7 are "Age", "Education", "Gender", "Working hours", "Marital", "Occupation", "Race" and "Workclass", respectively. Clearly, the average feature changes are consistent with the feature importance, e.g., "Age", "Education" and "Marital" are more important, and "Gender", "Race" and "Workclass" are less important.

(132) This experiment compares CERTIFAI, DiCE, the RL and GLD for generating diverse counterfactual examples per query instance. Algorithm 6 is applied for the RL and GLD. Since Greedy, SHAP-based and Foil trees cannot generate diverse counterfactuals, they are not compared here. Table 12 in FIG. **24** and Table 13 in FIG. **25** show the sparsity and diversity metrics. The RL and GLD methods obtain the lowest sparsity while DiCE achieves the highest diversity. Table 14 in FIG. **26** gives the running time for these methods. DiCE is much slower than CERTIFAI and the proposed methods. The proposed methods make a good trade-off between sparsity and diversity with much less running time per query. For practical applications, we can apply RL or GLD or both as ensemble. Table 10 in FIG. **22** and Table 11 in FIG. **23** give diverse counterfactual examples generated by the RL method, generating diverse explanations for the query instances.

(133) Additional experiment demonstrates that the proposed methods herein can also be applied to time series classification problems. The experiment is conducted on the Taxonomist dataset, where the query label is "ft" and we take the other labels as the desired labels and generate counterfactual examples for all the desired labels. The counterfactual examples are constructed by several time series (features), which is compared with Greedy since the other methods cannot handle time series data directly. Table 15 in FIG. **27** shows the sparsity and the running time for these methods, which shows that the proposed methods are more than 10 times faster than Greedy while achieve similar sparsity metrics for different setups. This result also shows that the proposed methods can handle high dimensional data efficiently.

(134) Some examples of computing devices, such as computing device **100** may include non-transitory, tangible, machine readable media that include executable code that when run by one or more processors (e.g., processor **110**) may cause the one or more processors to perform the processes of method **300**. Some common forms of machine readable media that may include the processes of method **300** are, for example, floppy disk, flexible disk, hard disk, magnetic tape, any other magnetic medium, CD-ROM, any other optical medium, punch cards, paper tape, any other physical medium with patterns of holes, RAM, PROM, EPROM, FLASH-EPROM, any other memory chip or cartridge, and/or any other medium from which a processor or computer is adapted to read.

(135) This description and the accompanying drawings that illustrate inventive aspects, embodiments, implementations, or applications should not be taken as limiting. Various mechanical, compositional, structural, electrical, and operational changes may be made without departing from the spirit and scope of this description and the claims. In some instances, well-known circuits, structures, or techniques have not been shown or described in detail in order not to obscure the embodiments of this disclosure Like numbers in two or more figures represent the same or similar elements.

(136) In this description, specific details are set forth describing some embodiments consistent with the present disclosure. Numerous specific details are set forth in order to provide a thorough

understanding of the embodiments. It will be apparent, however, to one skilled in the art that some embodiments may be practiced without some or all of these specific details. The specific embodiments disclosed herein are meant to be illustrative but not limiting. One skilled in the art may realize other elements that, although not specifically described here, are within the scope and the spirit of this disclosure. In addition, to avoid unnecessary repetition, one or more features shown and described in association with one embodiment may be incorporated into other embodiments unless specifically described otherwise or if the one or more features would make an embodiment non-functional.

(137) Although illustrative embodiments have been shown and described, a wide range of modification, change and substitution is contemplated in the foregoing disclosure and in some instances, some features of the embodiments may be employed without a corresponding use of other features. One of ordinary skill in the art would recognize many variations, alternatives, and modifications. Thus, the scope of the invention should be limited only by the following claims, and it is appropriate that the claims be construed broadly and in a manner consistent with the scope of the embodiments disclosed herein.

Claims

1. A method for generating a counterfactual example for counterfactual explanation in a machine learning model, the method comprising: receiving a query instance including a plurality of feature columns and corresponding feature values; generating, by a machine learning model, a prediction result in response to the query instance; identifying, from the plurality feature columns, a subset of feature columns associated with corresponding alternative feature values that would have potentially changed the prediction result; determining an optimal policy that maximizes an expected feedback reward earned for modifying the query instance including changing a feature value of at least one feature column of the subset of feature columns; determining a number of feature columns of the subset of feature columns and a number of associated feature values according to the determined optimal policy; constructing a counterfactual example using a portion of the determined number of feature columns and the number of associated feature values; and outputting, by the machine learning model, the counterfactual example as a counterfactual explanation for the prediction result in response to the query instance, wherein the optimal policy is determined by reinforcement learning (RL) in which: changing the feature value of the at least one feature column is treated as an action in the RL; the modified query instance is treated as a state in the RL; and a reward function of the RL for determining the expected feedback reward is a prediction score of a different label other than the prediction result.

2. The method of claim 1, wherein the optimal policy is determined subject to a sparsity constraint for selecting the action.

3. The method of claim 2, wherein the sparsity constraint is imposed via a probability distribution parameterized by parameters of the machine learning model.

4. The method of claim 1, wherein the optimal policy is determined by a gradientless descent procedure that encourages a sparse solution for selecting the action and a gradient descent procedure that encourages an exploration solution for selecting the action.

5. The method of claim 1, wherein: the counterfactual example is a first counterfactual example; and the constructing comprises constructing a diverse set of counterfactual examples including the first counterfactual example and a second counterfactual example using at least one feature column different from the portion of the determined number of feature columns.

6. The method of claim 5, further comprising: sorting the set of counterfactual examples based on respective objective function values; and removing duplicate counterfactual examples from the set of counterfactual examples.

7. The method of claim 1, wherein the machine learning model is a multivariate time series

classifier that receives the query instance of a time series of input sequences.
8. The method of claim 1, wherein the subset of feature columns are identified from a number of nearest neighboring query instances to the query instance via a nearest neighbor search tree built from a training dataset.
9. A system for generating a counterfactual example for counterfactual explanation in a machine learning model, the system comprising: a memory that stores the machine learning model; a data interface that receives a query instance including a plurality of feature columns and corresponding feature values; and a processor that reads instructions from the memory to perform: generating, by the machine learning model, a prediction result in response to the query instance; identifying, from the plurality feature columns, a subset of feature columns associated with corresponding alternative feature values that would have potentially changed the prediction result; determining an optimal policy that maximizes an expected feedback reward earned for modifying the query instance including changing a feature value of at least one feature column of the subset of feature columns; determining a number of feature columns of the subset of feature columns and a number of associated feature values according to the determined optimal policy; constructing a counterfactual example using a portion of the determined number of feature columns and the number of associated feature values; and outputting, by the machine learning model, the counterfactual example as a counterfactual explanation for the prediction result in response to the query instance, wherein the optimal policy is determined by reinforcement learning (RL) in which: changing the feature value of the at least one feature column is treated as an action in the RL; the modified query instance is treated as a state in the RL; and a reward function of the RL for determining the expected feedback reward is a prediction score of a different label other than the prediction result.
10. The system of claim 9, wherein the optimal policy is determined subject to a sparsity constraint that is imposed on a stochastic policy for selecting the action.
11. The system of claim 10, wherein the sparsity constraint is imposed via a probability distribution parameterized by parameters of the machine learning model.
12. The system of claim 9, wherein the optimal policy is determined by a gradientless descent procedure that encourages a sparse solution for selecting the action and a gradient descent procedure that encourages an exploration solution for selecting the action.
13. The system of claim 9, wherein the counterfactual example is a first counterfactual example and the processor further reads instructions from the memory to: construct a diverse set of counterfactual examples including the first counterfactual example and a second counterfactual example using at least one feature column different from the portion of the determined number of feature columns.
14. The system of claim 13, wherein the processor further reads instructions from the memory to perform: sorting the set of counterfactual examples based on respective objective function values; and removing duplicate counterfactual examples from the set of counterfactual examples.
15. The system of claim 9, wherein the machine learning model is a multivariate time series classifier that receives the query instance of a time series of input sequences.
16. A processor-readable non-transitory storage medium storing processor-readable instructions for generating a counterfactual example for counterfactual explanation in a machine learning model, the instructions being executed by a processor to perform: receiving a query instance including a plurality of feature columns and corresponding feature values; generating, by a machine learning model, a prediction result in response to the query instance; identifying, from the plurality feature columns, a subset of feature columns associated with corresponding alternative feature values that would have potentially changed the prediction result; determining an optimal policy that maximizes an expected feedback reward earned for modifying the query instance including changing a feature value of at least one feature column of the subset of feature columns; determining a number of feature columns of the subset of feature columns and a number of associated feature values according to the determined optimal policy; constructing a counterfactual example using a portion

of the determined number of feature columns and the number of associated feature values; and outputting, by the machine learning model, the counterfactual example as a counterfactual explanation for the prediction result in response to the query instance, wherein the optimal policy is determined by reinforcement learning (RL) in which: changing the feature value of the at least one feature column is treated as an action in the RL; the modified query instance is treated as a state in the RL; and a reward function of the RL for determining the expected feedback reward is a prediction score of a different label other than the prediction result.

17. The method of claim 1, further comprising selecting to output the counterfactual example as the counterfactual explanation based on its proximity to the query instance.

18. The method of claim 1, wherein: one of the determined number of feature columns is a continuous feature column; the constructing the counterfactual example includes refining the associated feature value of the continuous feature column using a gradientless descent procedure to optimize the proximity of the counterfactual explanation to the query instance; and the outputting the counterfactual example includes outputting the proximity-optimized counterfactual explanation as the counterfactual explanation for the prediction result in response to the query instance.

19. The processor-readable non-transitory storage medium of claim 16, wherein the optimal policy is determined subject to a sparsity constraint for selecting the action.

20. The processor-readable non-transitory storage medium of claim 16, wherein the subset of feature columns are identified from a number of nearest neighboring query instances to the query instance via a nearest neighbor search tree built from a training dataset.